CRLF Injection

CRLF Injection is the **root primitive** underneath several vulnerabilities you've already studied separately — it's less a single attack than a single *technique* that gets weaponized differently depending on what's parsing the injected bytes on the other end. Worth locking in the distinction immediately: this guide splits into **CRLF injection into HTTP contexts** (headers, redirects — directly extending your Response Splitting guide) and **CRLF injection into non-HTTP contexts** (log files, email headers, backend protocols) — same injected character sequence, completely different consequence depending on what interprets it. This connects directly back to your Response Splitting guide's Section 10-12 (which is, in fact, just the HTTP-specific instance of everything covered here) and your Log4Shell/injection-class discussions more broadly.

---

1. What CRLF Actually Is — The Foundation

`CRLF` is `\r\n` — Carriage Return (`0x0D`) followed by Line Feed (`0x0A`). It isn't inherently malicious; it's the **standard line-terminator** baked into an enormous number of text-based protocols and formats that predate the web:

```
HTTP headers        → separated by CRLF, message ends at a blank CRLF CRLF
SMTP (email)         → commands and headers separated by CRLF
Log file entries       → conventionally one CRLF (or LF) per entry
FTP / IMAP / POP3 / Redis / Memcached protocols
                       → all CRLF or LF delimited, command-based
```

The core vulnerability — the entire concept every variant below exploits: whenever an application takes **attacker-controlled input** and inserts it into one of these formats **without stripping or encoding the CRLF characters**, the attacker can terminate the current line/field early and **inject a new one** that the downstream parser will treat as fully legitimate, structurally separate content — not as part of the original field's value at all.

```
Intended structure:  [Header-Name]: [user input, ONE value]
Attacker input:      innocent-value\r\nInjected-Header: evil-value
Resulting structure: [Header-Name]: innocent-value
                     [Injected-Header]: evil-value      <- a NEW,
                                                           attacker-
                                                           controlled
                                                           field, never
```

```
                                                    intended by
                                                    the developer
```

This single fact — a parser trusting embedded line-terminators inside what should be an atomic value — is the entire root cause of every variant covered in this guide.

PART 1: CRLF Injection in HTTP Contexts

2. Where HTTP Reflects Input Into Headers

```
Common injection points:
- Redirect Location headers built from a query parameter
- Custom headers built from user input (X-Custom-Log, X-Debug-Info)
- Set-Cookie values built from user-controllable data
- Cache-related headers (Vary, Cache-Control) constructed dynamically
```

Any of these becomes exploitable the moment attacker input reaches the header-construction step **unsanitized** — this is exactly the reflection point your Response Splitting guide's Section 11 walked through in detail.

3. Encoding Matters — Literal vs Percent-Encoded CRLF

Browsers and modern frameworks are increasingly strict about rejecting **literal** raw `\r\n` bytes inside a URL or parameter. Attackers therefore test multiple encodings to find whichever layer in the chain fails to decode-then-reject correctly:

```
Literal:            \r\n
URL-encoded:        %0d%0a
Double URL-encoded: %250d%250a
Unicode variants:   %E5%98%8A (U+560A) / %E5%98%8D (U+560D)
                    - some parsers normalize these to CR/LF during
                      Unicode processing, bypassing filters that only
                      check for the literal ASCII sequence
```

Detection methodology:

```
1. Identify a reflected input point (query param, header, cookie)
2. Inject each encoding variant above followed by a distinctive
   fabricated header: X-Injected-Test: canary-value
3. View the RAW response (Burp Repeater / raw TCP - not a browser,
```

```
   since browsers normalize/reject this silently) to check whether
   your fabricated header appears as a genuine, separate header
4. If it does, you have a confirmed CRLF injection point - proceed
   to Section 4 for what to do with it
```

4. Escalating a Confirmed Injection — What It Enables

```
Single extra header injection  → set an arbitrary cookie
                                   (Set-Cookie: session=attacker-value)
                                   - session fixation, tracking
                                   manipulation
Security header suppression       → inject a header that OVERRIDES
                                      or duplicates a security-critical
                                      one (X-Frame-Options, CSP) that
                                      the server sends later, if the
                                      recipient honors the FIRST or
                                      LAST occurrence unexpectedly
Full HTTP Response Splitting        → inject a full blank-line-terminated
                                        second response (headers + body)
                                        - this is your Response Splitting
                                        guide's Section 11, in full
Cache poisoning via injected response → if an intermediate cache
                                          stores the fabricated
                                          "second response" under a
                                          normal cache key - direct
                                          convergence with your Cache
                                          Poisoning guide's delivery model
```

5. Prevention — HTTP-Context CRLF Injection

```
- NEVER build headers via raw string concatenation of user input -
  use the framework/language's structured header-setting API, which
  encodes or rejects embedded CR/LF automatically rather than
  passing them through
- Strip or reject CR and LF - both literal AND every encoded
  variant from Section 3 - from any value destined for a header,
  at the point of use, not just at the input boundary (a value
  sanitized on input can still be re-decoded later and reintroduce
  the raw bytes)
- Validate redirect targets against an allowlist instead of
  reflecting attacker-supplied URLs into Location headers directly
- Set explicit Cache-Control: private / no-store on responses
  containing any reflected, user-influenced header value, limiting
  the blast radius even if an injection slips through
```

PART 2: CRLF Injection in Non-HTTP Contexts

6. Log Injection / Log Forging

If user input is written into a log file without sanitization, an attacker can inject a CRLF followed by **fabricated log lines** that never actually happened:

```
Legitimate log format:
  [2026-07-28 10:15:32] LOGIN_FAILED user=alice

Attacker-supplied username field:
  alice\r\n[2026-07-28 10:15:33] LOGIN_SUCCESS user=admin

Resulting log file:
  [2026-07-28 10:15:32] LOGIN_FAILED user=alice
  [2026-07-28 10:15:33] LOGIN_SUCCESS user=admin   <- entirely
                                                      FABRICATED,
                                                      never actually
                                                      occurred
```

Why this matters beyond mere annoyance: logs are frequently trusted as forensic ground truth during an incident response investigation, fed into SIEM alerting rules, or parsed by automated tooling that takes action based on log content. A forged entry can **frame an innocent user, hide a real attack among fabricated noise, or trigger a downstream automated action** if the log pipeline parses and acts on log content (a genuinely underrated escalation path).

7. SMTP / Email Header Injection

Web applications that build outgoing emails from user input (a "contact us" form populating a `From`/`Subject` header, a password-reset email including a user-controlled display name) are exploitable the same way:

```
Legitimate intended header:
  Subject: Contact form submission

Attacker-supplied "name" field:
  Legit Name\r\nBcc: attacker@evil.com\r\nSubject: Fake Invoice

Resulting email headers:
  Subject: Contact form submission
  Bcc: attacker@evil.com          <- injected recipient, invisible
                                     to the visible "To" field
  Subject: Fake Invoice           <- SECOND Subject header, some
                                     mail clients honor the LAST
                                     one shown
```

This is a classic path to turning a legitimate transactional mail sender into a **spam relay** (adding arbitrary `Bcc`/`Cc` recipients at scale) or crafting **convincing phishing emails** that originate from the victim organization's own trusted mail infrastructure.

8. Protocol-Level Injection — Redis, Memcached, and Similar

Line-based backend protocols that an application talks to internally are equally vulnerable if user input is concatenated directly into a raw protocol command rather than passed through a proper client library's parameterized methods:

```
Intended command:  SET user:alice:session <value>
Attacker input for <value>:
  legit-value\r\nFLUSHALL\r\n
Resulting commands sent to Redis:
  SET user:alice:session legit-value
  FLUSHALL                                <- entirely separate,
                                             attacker-injected command,
                                             wipes the ENTIRE datastore
```

This variant is less commonly discussed than the HTTP/email cases but is arguably **more severe** in impact, since it grants direct command injection against a backend datastore rather than "just" header manipulation — worth treating any raw string-built protocol command as a smuggling-equivalent risk, structurally identical to the CL.TE/TE.CL discrepancies in your Request Smuggling guide even though no HTTP is involved at all.

9. Testing Methodology for Non-HTTP CRLF Injection

```
1. Identify every point where user input flows into: a log write,
   an outgoing email header, or a raw backend protocol command
   built via string concatenation rather than a parameterized
   client/library call
2. Inject a CRLF sequence followed by a distinctive fabricated
   entry/header/command (a canary log line, a canary Bcc address,
   a harmless backend command like PING rather than anything
   destructive during initial testing)
3. For logs: check whether the fabricated line appears as a
   SEPARATE entry in the log output, rather than folded into the
   original entry's value
4. For email: check the RAW email source (not just the rendered
   view) for the injected header
5. For backend protocols: check whether the injected command
   executed as a genuinely separate operation (use monitoring/
   read-only test commands FIRST - never test with destructive
```

```
   commands like FLUSHALL against anything but an isolated lab
   instance)
```

10. Prevention — Non-HTTP-Context CRLF Injection

```
- Use structured/parameterized logging libraries that escape or
  reject embedded CR/LF automatically, rather than raw string
  concatenation into a log line
- Use your mail library's structured header-setting API (never
  concatenate user input directly into raw SMTP header text) -
  reject CR/LF in any field destined for a header, especially
  From/Subject/Reply-To
- Use the backend datastore's PARAMETERIZED client methods
  (equivalent to prepared statements for SQL) rather than building
  raw protocol commands via string concatenation - this eliminates
  the injection surface structurally rather than trying to filter it
- Treat CRLF sanitization as a property of EVERY sink, not just the
  HTTP-facing ones - the same unsanitized input can often reach
  multiple sinks (a header AND a log line AND an email) from a
  single vulnerable field
```

11. Practical Lab Example

```
PortSwigger's Web Security Academy folds most HTTP-context CRLF
injection directly into its Response Splitting / cache poisoning
lab sets rather than treating it as fully standalone (see your
Response Splitting guide's Section 16). Log injection and email
header injection are less commonly lab-packaged but appear
frequently in OWASP's testing guide and in real-world bug bounty
writeups (search "CRLF injection Bcc" or "log forging" for concrete
disclosed examples).

Suggested practice order:
1. HTTP-context: repeat the Response Splitting guide's redirect-
   parameter lab (Section 3 above), this time explicitly testing
   EVERY encoding variant from Section 3, not just the literal one
2. Non-HTTP: set up a local sandbox app that writes a raw log line
   from a form field, and confirm you can forge a second log entry
   (Section 6) - the clearest, lowest-risk illustration of the
   non-HTTP variant
3. Non-HTTP: if comfortable with mail server setup, test the email
   header injection pattern (Section 7) against a local/test SMTP
   server only, never a real mail relay
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-93 (Improper Neutralization of CRLF Sequences - the umbrella entry for this entire guide), CWE-113 (HTTP Response Splitting, the HTTP-specific instance from Part 1), CWE-117 (Improper Output Neutralization for Logs, covering Section 6)
OWASP Top 10:2025	A05:2025 Injection (the umbrella category for all variants in this guide, since every case is unsanitized input reaching a structurally-sensitive sink)
CVSS	HTTP-context injection severity scales with what's reachable downstream (Medium for a suppressed security header, High/Critical when chained into cache poisoning per Part 1 Section 4); log/email injection is typically Medium alone but escalates sharply if a SIEM/automation pipeline acts on forged log content, or if email injection enables large-scale spam relay abuse
Cyber Kill Chain	Delivery (the injected content itself is the delivery mechanism, whether it's a forged log line, a phishing email, or a split HTTP response) → Exploitation / Actions on Objectives (whatever downstream system trusts and acts on the forged content)
MITRE ATT&CK	T1562.006 (Indicator Blocking / log manipulation, for Section 6) T1585.002 (Email Accounts, when injection is used to establish spoofed sender infrastructure for Section 7)

Quick Reference Cheat Sheet

```
HTTP-CONTEXT CRLF INJECTION (inject into a response HEADER):
1. Find user input reflected into a header (redirect, custom
   header, Set-Cookie)
2. Inject CRLF - literal, then %0d%0a, then Unicode variants if
   filtered - plus a fabricated header
3. Confirm via RAW response view (not browser) that the fabricated
   header/response is recognized
4. Weaponize: session fixation, security-header suppression, full
   response splitting, or cache poisoning (see Response Splitting
   guide)

NON-HTTP CRLF INJECTION (inject into a LOG / EMAIL / BACKEND
PROTOCOL sink):
1. Find user input written raw into a log line, email header, or
   backend protocol command
2. Inject CRLF plus a fabricated log entry / header / command
3. Confirm the fabricated content appears as a genuinely SEPARATE
   entry/header/command, not folded into the original field
```

```
4. Weaponize: log forgery for forensic evasion, spam-relay/phishing
   via injected Bcc/Subject, or direct backend command execution

Root cause shared by both: a parser trusting an embedded
line-terminator inside what the developer assumed was an atomic,
single-line value - identical underlying primitive, different sink
```
